

CSE335 OPERATING SYSTEMS PROJECT

Scheduling, Hierarchical Paging, and Extent-Aware Storage in MINIX 3.3.0

Design, implementation, verification, experimental method, and analysis

Course	Prepared by	Term	Platform
CSE335	Rodina and project team	Summer 2026	MINIX 3.3.0

Submission note: the source, native builds, deterministic tests, and all three experiment matrices were completed inside MINIX 3.3.0 on 18 August 2026. The unedited CSV files and validation log are included under project-deliverables/native-results.

Executive summary

This project extends a clean MINIX 3.3.0 source tree with three controlled operating-systems experiments. The first compares Round Robin, shortest-job-first, static priority, and multilevel feedback queue scheduling using real child processes. The second constructs configurable hierarchical page tables, drives them with byte-address traces, and compares FIFO and LRU replacement. The third investigates MFS bitmap allocation, modifies its real zone-selection path to prefer user-defined contiguous runs, and benchmarks file and directory operations on a scratch MFS file system. Every experiment uses a text configuration file and produces CSV output instead of relying on screenshots or hand-copied numbers.

The design deliberately separates hardware facts from experimental variables. On x86, the running MINIX kernel cannot change page size or page-table depth merely because a user configuration file requests it. Requirement 2 therefore models the requested hierarchy inside a native MINIX process. The unreliable legacy VM_INFO telemetry call is not linked. In contrast, Requirement 3 changes the production MFS allocation path because extent placement can be altered without changing the on-disk format. Requirement 1 uses real processes but keeps policy decisions in a controlled dispatcher.

The repository is organized around reproducibility. The untouched release is tagged baseline-v3.3.0; requirement-focused commits follow it. All three known-workload tests passed natively. The archived matrices contain 80 scheduling rows, 72 paging rows, and 18 extent rows. Every extent row reported zero verification errors, and MFS logged six preferred-run hits with zero fallbacks for each requested size from 2 through 32 zones.

1. Problem interpretation and project boundaries

The project specification asks for experiments rather than isolated textbook descriptions. Each requirement combines implementation, configuration, measurement, and explanation. A compliant solution therefore needs four layers: a mechanism that actually performs the work, a policy that can be varied, instrumentation that records comparable outcomes, and a documented procedure that another student or instructor can repeat. The implementation follows that structure consistently. The command directories contain parsers, algorithms, and tests; the etc directory contains editable defaults; CSV output captures raw observations; and this report turns observations into conclusions without hiding uncertainty.

Starting from the clean 3.3.0 release was essential. The current public MINIX repository has continued beyond the historical release and its default branch is not a drop-in substitute for a 3.3.0 assignment image. Mixing files from a later tree with a 3.3.0 VM can fail because server interfaces, build files,

headers, or library contracts differ. The submitted repository therefore records the exact baseline before any project code. This makes every modification reviewable with `git diff` and prevents a copied GitHub implementation from silently targeting a different system version.

A second boundary concerns evidence. The authoritative gate was compilation and execution inside the bootable MINIX 3.3 virtual machine. The three commands and the modified MFS server built with the native toolchain; all deterministic tests passed; and the three raw matrices and MFS console counters were copied back unchanged. The validation log records the OS identity, test output, row counts, checksums, and allocator messages.

A third boundary concerns system safety. Replacing a root file-system server or formatting an unidentified disk can destroy the working VM. The extent experiment is therefore designed for a disposable secondary MFS device, ideally after taking a VM snapshot. It retains the existing bitmap allocator as the only function that marks zones allocated, changes no on-disk structures, and falls back to ordinary allocation when a requested run is unavailable. The benchmark deletes only its own per-run subdirectories and data files.

2. Baseline architecture

Subsystem	Stock MINIX responsibility	Project intervention
Scheduling	Kernel scheduling plus scheduling server control	Real-process dispatcher comparing four logical policies
Virtual memory	VM server, hardware mappings, fixed platform page geometry	Configurable software hierarchy and FIFO/LRU trace experiment
MFS	Inodes, zone bitmap, block cache, file and directory operations	Exact-origin allocation and free-run preference in real MFS

MINIX uses a microkernel-oriented architecture in which services that monolithic systems often place in one privileged kernel are separated. This matters to the project because scheduling, virtual memory, and file-system behavior do not live in one source file or one protection domain. The experiment commands are ordinary user processes; VM information is obtained through the published `libsys` interface; and MFS is a restartable file-system service. The design respects those boundaries rather than pretending a portable user program can alter x86 translation hardware or directly edit live file-system metadata.

MFS free-space management is bitmap based. An inode describes a file and stores direct and indirect zone references. When a write needs a new zone, `alloc_zone` converts a preferred physical zone to a bitmap origin and `alloc_bit` scans the zone map for a zero bit, sets it, marks the bitmap buffer dirty, and

updates accounting. `free_zone` performs the reverse operation. Directories are files whose records associate names with inode numbers; path lookup resolves those records, while `create`, `read`, `write`, and `unlink` eventually interact with inode and zone operations. This call chain identifies `alloc_zone` and `alloc_bit` as the narrow, defensible intervention point for an extent preference.

The production MINIX scheduler is not replaced by `schedexperiment`. The experiment creates real workers, but a parent process decides which worker may run a logical slice. This produces deterministic policy metrics and executes genuine CPU work. Similarly, `vmexperiment` does not claim that a configurable structure becomes the process's hardware page table. It constructs a sparse hierarchy that records visited indices for each virtual page and drives replacement frames. Keeping these claims precise makes the analysis technically credible.

3. Requirement 1 — scheduling experiment

3.1 Workload and process model

The scheduler configuration contains an algorithm selector, Round Robin quantum, three MLFQ quanta, a CPU-work scale, an output path, and repeated process records. Each process record supplies a name, logical arrival time, burst length, and static priority. Smaller priority numbers represent higher priority. The default workload intentionally contains different burst lengths, staggered arrivals, and a priority order that disagrees with the shortest-job order. That prevents two algorithms from producing identical schedules merely because the input was too simple.

At startup the parent creates two pipes and forks once per configured job. The child blocks waiting for a command containing the number of logical milliseconds it may execute. It then performs deterministic integer arithmetic for that many units and returns an acknowledgement. The parent never records a slice as completed until the matching child replies. All children are genuine MINIX processes, but logical arrival controls eligibility rather than fork time. This distinction allows repeatable calculations while still measuring the wall time consumed by real work.

Logical time is the sum of dispatched burst slices plus any idle jump to the next arrival. It is independent of host speed, VM load, and compiler optimization. Wall elapsed time is recorded separately and is useful for checking that work took place, but it is not used to calculate turnaround or waiting. This two-clock design avoids confusing policy behavior with incidental virtualization overhead. It also makes a known workload suitable for an exact regression test.

3.2 Algorithms

Round Robin. The implementation scans ready jobs from a rotating cursor and dispatches at most `quantum_ms`. Its principal advantage is strong response and fairness, but more dispatches as the quantum shrinks. The main caveat is that a large quantum approaches first-come first-served behavior. Ties are resolved deterministically using arrival order or queue stamps, which prevents results from changing because of array traversal accidents. A dispatch increments the context-switch proxy even if the same worker would be selected again; the metric therefore represents controlled hand-offs rather than exact kernel context switches.

SJF. The implementation selects the ready job with the smallest remaining burst and runs it to completion. Its principal advantage is low mean waiting for a known workload. The main caveat is that future burst length is assumed and long jobs can wait. Ties are resolved deterministically using arrival order or queue stamps, which prevents results from changing because of array traversal accidents. A dispatch increments the context-switch proxy even if the same worker would be selected again; the metric therefore represents controlled hand-offs rather than exact kernel context switches.

Static priority. The implementation selects the smallest numeric priority among ready jobs and runs it to completion. Its principal advantage is expresses importance directly. The main caveat is that low-priority work may starve without aging. Ties are resolved deterministically using arrival order or queue stamps, which prevents results from changing because of array traversal accidents. A dispatch increments the context-switch proxy even if the same worker would be selected again; the metric therefore represents controlled hand-offs rather than exact kernel context switches.

MLFQ. The implementation starts every job in queue zero, uses increasing quanta, and demotes incomplete jobs. Its principal advantage is approximates short-job preference without supplied burst estimates. The main caveat is that its behavior depends strongly on queue count, quanta, and boost policy. Ties are resolved deterministically using arrival order or queue stamps, which prevents results from changing because of array traversal accidents. A dispatch increments the context-switch proxy even if the same worker would be selected again; the metric therefore represents controlled hand-offs rather than exact kernel context switches.

3.3 Native known-workload results

Algorithm	Avg turnaround (ms)	Avg waiting (ms)	Avg response (ms)	Makespan	Dispatches
RR	185.00	133.00	17.00	260	14
SJF	137.00	85.00	85.00	260	5
PRIORITY	155.00	103.00	103.00	260	5
MLFQ	191.00	139.00	5.00	260	15

Table 1 was produced and checked inside MINIX by the included known.conf test. The makespan is identical because all policies eventually execute the same 260 logical milliseconds and the workload has no idle gap after time zero. The differences are distributional: SJF minimizes average waiting for this workload, while MLFQ gives the quickest average first response at the cost of additional dispatches.

The experimental matrix varies the base quantum through 5, 10, 20, and 40 milliseconds while holding arrivals, bursts, priorities, CPU-work scale, and MLFQ ratios constant. RR and MLFQ should change as the quantum changes; non-preemptive SJF and priority should not change logically, although their measured wall time can fluctuate. This provides an internal control: if SJF logical averages change across those rows, either parsing, matrix construction, or result aggregation is wrong.

4. Requirement 2 — hierarchical paging

4.1 Address translation model

The paging configuration defines address width, power-of-two page size, hierarchy level count, one index width per level, frame count, replacement policy, and trace generator. Validation requires the sum of all hierarchy index bits plus the page-offset bits to equal the configured address width. For a 32-bit address, a 4 KiB page has a 12-bit offset; a two-level 10/10 split therefore consumes the remaining 20 bits. Changing to an 8 KiB page requires a 13-bit offset and a compatible 19-bit hierarchy split.

The hierarchy is sparse. A reference first divides the byte address by page size to obtain a virtual page number. From the highest configured level to the lowest, masks and shifts extract an index. Nodes and entries are allocated only for paths that appear in the trace. A leaf records the page identity used by replacement. This structure models the memory overhead advantage of hierarchical tables: a flat table conceptually provides every possible leaf, while a sparse hierarchy pays for directories and tables that are actually touched.

The trace is expressed in byte addresses so a page-size comparison keeps the underlying byte workload constant. Sequential mode advances by a fixed byte stride; locality mode selects most accesses from a hot byte region and the remainder from a larger working set; random mode samples the working set uniformly; file mode reads explicit decimal or hexadecimal addresses. A fixed seed makes generated traces repeatable. This design avoids the common error of generating page numbers and then silently changing the amount of memory represented when page size changes.

4.2 FIFO and LRU

Both policies share the same frame table and trace. On a hit, FIFO leaves arrival order unchanged; LRU updates the frame's last-use counter. On a miss with an unused frame, the page enters that frame and the empty-frame counter decreases. Once all frames are occupied, FIFO replaces the frame with the oldest load stamp, whereas LRU replaces the least recently accessed frame. Faults include first fills and replacements; replacements exclude first fills. This distinction allows the results to show both pressure and initial capacity consumption.

FIFO is simple and requires little recency bookkeeping, but it can evict a heavily used page merely because that page arrived early. It can also exhibit Belady's anomaly for some reference strings, where adding frames increases faults. LRU uses recency as a locality heuristic and is a stack algorithm, so ideal LRU does not have that anomaly. Exact LRU is more expensive in a real operating system; practical kernels approximate it with reference bits, aging, or clock families. The experiment implements exact LRU because the purpose is controlled policy comparison rather than production overhead optimization.

A deterministic test uses a small, known address trace and compares exact FIFO and LRU fault totals. It catches errors in hit detection, victim choice, frame initialization, and page-size conversion. The matrix script then varies page size and level split while holding the byte trace definition and number of frames constant. Result rows include page-table nodes, entries, and estimated bytes, letting the analysis discuss a trade-off that fault totals alone would miss.

4.3 Native MINIX results and interpretation

Page size	FIFO faults	LRU faults	Status
1,024 B	6,937	6,791	Measured in MINIX
2,048 B	4,735	4,047	Measured in MINIX
4,096 B	2,666	1,655	Measured in MINIX
8,192 B	1,378	1,064	Measured in MINIX

These values are from the native 72-row matrix for the deterministic locality workload with 64 simulated frames. Faults decline as page size increases because each frame covers more of the fixed byte working set. That observation does not prove that the largest page is universally best: larger pages can increase internal fragmentation, copy unused bytes, reduce granularity, and change table memory. The project records hierarchy allocation so the conclusion remains multidimensional.

The legacy MINIX 3.3 VM_INFO request did not return reliably on this image and was therefore removed from the final executable. The CSV retains the real-context columns as NA and keeps all simulated metrics explicit. This limitation is recorded rather than presenting simulator faults as hardware page faults.

5. Requirement 3 — extent-aware MFS allocation

5.1 Stock free-space path

MFS tracks free inodes and zones with bitmaps stored in file-system blocks. A zero zone-map bit represents a free data zone; setting it allocates the zone. `alloc_zone` receives a preferred physical zone, converts it to the map's numbering convention, and asks `alloc_bit` for one free bit. `alloc_bit` selects a map block and bitmap word from an origin, scans for a zero, writes the changed word in native byte order, marks the cache buffer dirty, and updates block accounting. `free_zone` clears the corresponding bit and moves the search hint backward when appropriate.

The original allocator used the origin to choose the containing word but began scanning at bit zero of that word. That coarseness prevents an extent search from selecting an exact run start when the start lies inside a bitmap word. The modification adds `first_bit`, begins the first word at origin modulo the bitmap-word width, then resets the offset after moving to later words. All original wrapping, bounds checking, dirty marking, and accounting remain in place.

For the first allocation of a file, the modified `alloc_zone` scans the zone bitmap from the preferred origin for a sequence of free bits as long as `mfs_extent_size`. If it finds a run, that first bit becomes the exact `alloc_bit` origin. If it does not, the code increments a fallback counter and calls the original allocation path using the normal preference. The scan itself is read-only: it never reserves zones. Normal later allocations use the zone following the previous one as their preference, so they consume the selected free run while it stays available.

5.2 Configuration, consistency, and limitations

The MFS service reads `mfs_extent_size` from its startup arguments through `env_parse`, with an allowed range of one through 1,024 zones and a default of one. A value of one reproduces ordinary behavior.

MINIX mount passes `-o` options to a newly started file-system service, so a scratch device can be mounted with `mount -t mfs -o mfs_extent_size=8 device mountpoint`. The benchmark's `extent_blocks` must match the mounted service preference for a controlled run. The user-facing `/etc/extent.conf` configures the benchmark; the mount option configures MFS itself.

Not reserving all zones in advance is a deliberate consistency decision. A reservation invisible to inode mappings would leak space after a crash or require new on-disk metadata and recovery rules. The implemented policy instead chooses a favorable start and lets the proven allocator mark each zone exactly when MFS assigns it. A concurrent allocation could consume part of the discovered run before the file grows, so the preference is not a hard guarantee. MFS request serialization and a quiet scratch file system reduce that risk during experiments. The report describes the feature as extent-biased allocation, not as a new persistent extent map.

The change preserves file format compatibility. Existing direct, single-indirect, and double-indirect zone references continue to represent file blocks; `fsck` and `mkfs` do not require changes. Freeing remains per zone. The modification affects placement and instrumentation only. At unmount, MFS prints requested size, search count, hit count, and fallback count. Those counters help distinguish a slow benchmark caused by fragmented free space from one in which the requested run was usually found.

5.3 Real file and directory benchmark

`extentexperiment` creates a base directory if needed and a unique subdirectory for each iteration. It creates and opens `data.bin`, writes a deterministic byte pattern in chunks equal to `extent_blocks` multiplied by `block_size`, calls `fsync`, records file allocation information with `fstat`, seeks to the beginning, reads every byte, verifies the pattern, closes the descriptor, unlinks the file, and removes the subdirectory. This sequence exercises directory entry creation, inode creation, zone allocation, cache writes, persistence, lookup, reads, deallocation, and directory removal.

CSV fields record the configured extent size, iteration, block and file sizes, total bytes, logical extent count, allocated 512-byte blocks, create/write/read/remove microseconds, calculated write and read MiB/s, and verification errors. Throughput is bytes divided by measured operation time. Logical extent count is the intended chunk grouping rather than a claim that MFS stores an extent tree. A zero verification count is mandatory; fast results with corrupted content are failures.

The matrix uses extent preferences of 1, 2, 4, 8, 16, and 32 blocks with three repetitions each while holding block size and file size constant. Each preference should be applied to a fresh mount of the scratch MFS device. For strongest control, restore or recreate the same file-system image before each preference, minimize background activity, discard the first warm-up run or report it separately, and

summarize the median plus range. Without remounting MFS, changing only extent.conf changes write chunk size but not the allocator preference, which would confound the experiment.

Requested zones	Searches	Preferred-run hits	Fallbacks
1	0	0	0
2	6	6	0
4	6	6	0
8	6	6	0
16	6	6	0
32	6	6	0

The real MFS matrix used a disposable 64 MiB image on /dev/vnd0 and remounted it for every preference. It produced 18 benchmark rows and zero verification errors. The 3.3 timer is coarse enough that some short operations round to zero microseconds, so allocator hit/fallback counters and data correctness are stronger evidence than small throughput differences in this run.

6. Integrated experimental method

Stage	Action
Preserve	Take a VM snapshot and confirm that /usr/src corresponds to the submitted tree and baseline tag.
Build	Compile and install the three experiment commands, then compile and install MFS using the VM's native toolchain.
Smoke-test	Run every known-workload shell test before collecting a matrix; stop if any expected value or data verification fails.
Control	Change one principal independent variable at a time and keep workloads, seeds, frames, file sizes, and environment stable.
Measure	Write raw CSV directly from the program and capture MFS console counters; do not transcribe values manually.
Repeat	Use repeated extent trials and, where time allows, repeated wall-time scheduling trials to expose VM noise.
Analyze	Compute comparable summaries, label oracle versus measured data, and interpret both benefits and costs.
Archive	Copy CSV and console evidence out of the VM, record the commit hash, and create the final source archive.

Independent variables are scheduler policy and quantum, simulated page size/hierarchy/replacement, and MFS extent preference. Dependent variables are turnaround, waiting, response, dispatches, faults, hits, replacements, empty frames, hierarchy memory, allocation hits/fallbacks, operation latency, throughput, and verification errors. Controlled variables include process workload, trace seed and byte working set, frame count, benchmark file size, scratch device state, compiler tree, and VM resources.

Threats to internal validity include background VM activity, cache warmth, compiler differences, an extent configuration that does not match the mount option, and running matrices against different disk states. Threats to construct validity include treating logical dispatches as kernel context switches, treating simulated faults as hardware page faults, or treating write chunk count as an on-disk extent tree. Threats to external validity include the small synthetic workloads and MINIX's educational architecture. Each reported conclusion must remain within those boundaries.

The analysis should prefer comparisons and mechanisms over a single winner. SJF can minimize an average yet harm responsiveness; MLFQ can respond quickly yet add dispatch cost. LRU can reduce faults yet require more bookkeeping. Larger pages can reduce faults for a localized byte workload yet waste space. Larger extent preferences can improve sequential placement when runs exist yet spend more time scanning a fragmented bitmap. Operating-system policy is a trade-off surface, not a ranking detached from workload.

7. Build, test, and run procedure

```
cd /usr/src && make includes
cd /usr/src/minix/commands/schedexperiment && make && make install
cd /usr/src/minix/commands/vmexperiment && make && make install
cd /usr/src/minix/commands/extentexperiment && make && make install
cp /usr/src/etc/scheduler.conf /etc/scheduler.conf
cp /usr/src/etc/paging.conf /etc/paging.conf
cp /usr/src/etc/extent.conf /etc/extent.conf
cd /usr/src/minix/fs/mfs && make && make install
cd /usr/src/minix/commands/schedexperiment/tests && sh test_known.sh
cd /usr/src/minix/commands/vmexperiment/tests && sh test_known.sh
cd /usr/src/minix/commands/extentexperiment/tests && sh test_known.sh
run_schedexperiments.sh /tmp/scheduling-matrix.csv
run_vmexperiments.sh /tmp/paging-matrix.csv
run_extent_matrix.sh /tmp/extent-matrix.csv
```

The MFS command must be tested on a secondary device. First verify the device identity using the VM's disk and mount information. Create or use an already prepared scratch MFS file system only after that verification, create /mnt/extenttest, and mount with the desired mfs_extent_size option. Point extent.conf at a directory under that mount. Unmount after each matrix segment so MFS prints its counters and the next extent preference starts in a new server instance.

A failed compile should be handled as evidence, not bypassed. Record the first compiler diagnostic, inspect the exact source and header contract, fix the source in the repository, recommit, and rebuild from a clean command directory. A failed deterministic test should stop performance collection because later numbers would be meaningless. A nonzero `verify_errors` value is always a correctness failure regardless of throughput.

8. Source-level audit

`schedexperiment/config.c`

Role and rationale. This unit parses and validates policy, quanta, worker scale, output, and process rows. It is kept narrowly scoped so a reviewer can connect the requirement to a small number of functions instead of searching across unrelated MINIX subsystems. Its output or state feeds a documented CSV column, console counter, or downstream correctness check.

Critical review point. The implementation must reject malformed workloads and bounds before forking. The relevant failure mode is treated as a correctness problem rather than hidden by a performance average. Error returns carry a concise diagnostic and callers stop the current experiment, preserving the raw evidence needed for debugging.

Verification evidence. The primary check is `known.conf` plus `invalid-input` checks. Review should compare configuration, console output, and CSV together, then inspect the git diff against `baseline-v3.3.0`. This triangulation checks both what the source intends and what the MINIX runtime actually executed.

`schedexperiment/scheduler.c`

Role and rationale. This unit creates workers and implements four selectors and metric accounting. It is kept narrowly scoped so a reviewer can connect the requirement to a small number of functions instead of searching across unrelated MINIX subsystems. Its output or state feeds a documented CSV column, console counter, or downstream correctness check.

Critical review point. The implementation must pipe/fork failure cleanup and deterministic ties. The relevant failure mode is treated as a correctness problem rather than hidden by a performance average. Error returns carry a concise diagnostic and callers stop the current experiment, preserving the raw evidence needed for debugging.

Verification evidence. The primary check is `known averages` and `normal child exit`. Review should compare configuration, console output, and CSV together, then inspect the git diff against `baseline-`

v3.3.0. This triangulation checks both what the source intends and what the MINIX runtime actually executed.

`schedexperiment/schedexperiment.c`

Role and rationale. This unit drives selected algorithms and writes per-job CSV. It is kept narrowly scoped so a reviewer can connect the requirement to a small number of functions instead of searching across unrelated MINIX subsystems. Its output or state feeds a documented CSV column, console counter, or downstream correctness check.

Critical review point. The implementation must never mix a partial failed run into output. The relevant failure mode is treated as a correctness problem rather than hidden by a performance average. Error returns carry a concise diagnostic and callers stop the current experiment, preserving the raw evidence needed for debugging.

Verification evidence. The primary check is header and row-count inspection. Review should compare configuration, console output, and CSV together, then inspect the git diff against baseline-v3.3.0. This triangulation checks both what the source intends and what the MINIX runtime actually executed.

`run_schedexperiments.sh`

Role and rationale. This unit varies base quantum while preserving workload. It is kept narrowly scoped so a reviewer can connect the requirement to a small number of functions instead of searching across unrelated MINIX subsystems. Its output or state feeds a documented CSV column, console counter, or downstream correctness check.

Critical review point. The implementation must add quantum as an explicit matrix column. The relevant failure mode is treated as a correctness problem rather than hidden by a performance average. Error returns carry a concise diagnostic and callers stop the current experiment, preserving the raw evidence needed for debugging.

Verification evidence. The primary check is four-quantum CSV audit. Review should compare configuration, console output, and CSV together, then inspect the git diff against baseline-v3.3.0. This triangulation checks both what the source intends and what the MINIX runtime actually executed.

`vmexperiment/config.c`

Role and rationale. This unit parses address geometry, frames, policy, and trace definition. It is kept narrowly scoped so a reviewer can connect the requirement to a small number of functions instead of searching across unrelated MINIX subsystems. Its output or state feeds a documented CSV column, console counter, or downstream correctness check.

Critical review point. The implementation must enforce exact bit sum and power-of-two page size. The relevant failure mode is treated as a correctness problem rather than hidden by a performance average. Error returns carry a concise diagnostic and callers stop the current experiment, preserving the raw evidence needed for debugging.

Verification evidence. The primary check is valid and deliberately invalid configurations. Review should compare configuration, console output, and CSV together, then inspect the git diff against baseline-v3.3.0. This triangulation checks both what the source intends and what the MINIX runtime actually executed.

vmexperiment/simulator.c

Role and rationale. This unit builds sparse page paths, traces, frames, FIFO, and LRU. It is kept narrowly scoped so a reviewer can connect the requirement to a small number of functions instead of searching across unrelated MINIX subsystems. Its output or state feeds a documented CSV column, console counter, or downstream correctness check.

Critical review point. The implementation must use byte addresses and deterministic stamps. The relevant failure mode is treated as a correctness problem rather than hidden by a performance average. Error returns carry a concise diagnostic and callers stop the current experiment, preserving the raw evidence needed for debugging.

Verification evidence. The primary check is known reference-string oracle. Review should compare configuration, console output, and CSV together, then inspect the git diff against baseline-v3.3.0. This triangulation checks both what the source intends and what the MINIX runtime actually executed.

vmexperiment/vmexperiment.c

Role and rationale. This unit runs both policies and writes distinct simulated metrics. It is kept narrowly scoped so a reviewer can connect the requirement to a small number of functions instead of searching across unrelated MINIX subsystems. Its output or state feeds a documented CSV column, console counter, or downstream correctness check.

Critical review point. The implementation must do not claim simulator faults are hardware faults. The relevant failure mode is treated as a correctness problem rather than hidden by a performance average. Error returns carry a concise diagnostic and callers stop the current experiment, preserving the raw evidence needed for debugging.

Verification evidence. The primary check is native known trace plus 72-row matrix. Review should compare configuration, console output, and CSV together, then inspect the git diff against baseline-

v3.3.0. This triangulation checks both what the source intends and what the MINIX runtime actually executed.

run_vmexperiments.sh

Role and rationale. This unit generates controlled page-size and hierarchy cases. It is kept narrowly scoped so a reviewer can connect the requirement to a small number of functions instead of searching across unrelated MINIX subsystems. Its output or state feeds a documented CSV column, console counter, or downstream correctness check.

Critical review point. The implementation must hold byte workload and frames constant. The relevant failure mode is treated as a correctness problem rather than hidden by a performance average. Error returns carry a concise diagnostic and callers stop the current experiment, preserving the raw evidence needed for debugging.

Verification evidence. The primary check is matrix row and configuration review. Review should compare configuration, console output, and CSV together, then inspect the git diff against baseline-v3.3.0. This triangulation checks both what the source intends and what the MINIX runtime actually executed.

mfs/cache.c

Role and rationale. This unit finds a free zone run and biases first-zone allocation. It is kept narrowly scoped so a reviewer can connect the requirement to a small number of functions instead of searching across unrelated MINIX subsystems. Its output or state feeds a documented CSV column, console counter, or downstream correctness check.

Critical review point. The implementation must scan read-only and retain safe fallback. The relevant failure mode is treated as a correctness problem rather than hidden by a performance average. Error returns carry a concise diagnostic and callers stop the current experiment, preserving the raw evidence needed for debugging.

Verification evidence. The primary check is mount counters plus file verification. Review should compare configuration, console output, and CSV together, then inspect the git diff against baseline-v3.3.0. This triangulation checks both what the source intends and what the MINIX runtime actually executed.

mfs/super.c

Role and rationale. This unit honors the exact origin bit inside the first bitmap word. It is kept narrowly scoped so a reviewer can connect the requirement to a small number of functions instead of searching across unrelated MINIX subsystems. Its output or state feeds a documented CSV column, console counter, or downstream correctness check.

Critical review point. The implementation must reset offset after the first word and preserve wrap bounds. The relevant failure mode is treated as a correctness problem rather than hidden by a performance average. Error returns carry a concise diagnostic and callers stop the current experiment, preserving the raw evidence needed for debugging.

Verification evidence. The primary check is fragmented scratch-image allocation cases. Review should compare configuration, console output, and CSV together, then inspect the git diff against baseline-v3.3.0. This triangulation checks both what the source intends and what the MINIX runtime actually executed.

mfs/main.c

Role and rationale. This unit reads bounded extent preference at service startup. It is kept narrowly scoped so a reviewer can connect the requirement to a small number of functions instead of searching across unrelated MINIX subsystems. Its output or state feeds a documented CSV column, console counter, or downstream correctness check.

Critical review point. The implementation must default one preserves compatibility. The relevant failure mode is treated as a correctness problem rather than hidden by a performance average. Error returns carry a concise diagnostic and callers stop the current experiment, preserving the raw evidence needed for debugging.

Verification evidence. The primary check is server startup log. Review should compare configuration, console output, and CSV together, then inspect the git diff against baseline-v3.3.0. This triangulation checks both what the source intends and what the MINIX runtime actually executed.

mfs/mount.c

Role and rationale. This unit reports search, hit, and fallback counters at unmount. It is kept narrowly scoped so a reviewer can connect the requirement to a small number of functions instead of searching across unrelated MINIX subsystems. Its output or state feeds a documented CSV column, console counter, or downstream correctness check.

Critical review point. The implementation must report before resetting mounted state. The relevant failure mode is treated as a correctness problem rather than hidden by a performance average. Error returns carry a concise diagnostic and callers stop the current experiment, preserving the raw evidence needed for debugging.

Verification evidence. The primary check is captured console output. Review should compare configuration, console output, and CSV together, then inspect the git diff against baseline-v3.3.0. This triangulation checks both what the source intends and what the MINIX runtime actually executed.

extentexperiment/config.c

Role and rationale. This unit parses sizes, repetitions, paths, and safety limits. It is kept narrowly scoped so a reviewer can connect the requirement to a small number of functions instead of searching across unrelated MINIX subsystems. Its output or state feeds a documented CSV column, console counter, or downstream correctness check.

Critical review point. The implementation must prevent overflow and excessive buffers/files. The relevant failure mode is treated as a correctness problem rather than hidden by a performance average. Error returns carry a concise diagnostic and callers stop the current experiment, preserving the raw evidence needed for debugging.

Verification evidence. The primary check is known.conf and boundary inputs. Review should compare configuration, console output, and CSV together, then inspect the git diff against baseline-v3.3.0. This triangulation checks both what the source intends and what the MINIX runtime actually executed.

extentexperiment.c

Role and rationale. This unit performs real directory and verified file I/O. It is kept narrowly scoped so a reviewer can connect the requirement to a small number of functions instead of searching across unrelated MINIX subsystems. Its output or state feeds a documented CSV column, console counter, or downstream correctness check.

Critical review point. The implementation must clean up only owned paths on every error. The relevant failure mode is treated as a correctness problem rather than hidden by a performance average. Error returns carry a concise diagnostic and callers stop the current experiment, preserving the raw evidence needed for debugging.

Verification evidence. The primary check is two-row smoke test with zero errors. Review should compare configuration, console output, and CSV together, then inspect the git diff against baseline-v3.3.0. This triangulation checks both what the source intends and what the MINIX runtime actually executed.

run_extent_matrix.sh

Role and rationale. This unit remounts a scratch MFS device and collects six extent sizes. It is kept narrowly scoped so a reviewer can connect the requirement to a small number of functions instead of searching across unrelated MINIX subsystems. Its output or state feeds a documented CSV column, console counter, or downstream correctness check.

Critical review point. The implementation must never target a system filesystem. The relevant failure mode is treated as a correctness problem rather than hidden by a performance average. Error returns

carry a concise diagnostic and callers stop the current experiment, preserving the raw evidence needed for debugging.

Verification evidence. The primary check is 18 measured rows plus MFS counters. Review should compare configuration, console output, and CSV together, then inspect the git diff against baseline-v3.3.0. This triangulation checks both what the source intends and what the MINIX runtime actually executed.

etc/*.conf

Role and rationale. This unit provides editable, documented defaults. It is kept narrowly scoped so a reviewer can connect the requirement to a small number of functions instead of searching across unrelated MINIX subsystems. Its output or state feeds a documented CSV column, console counter, or downstream correctness check.

Critical review point. The implementation must separate service option from benchmark option. The relevant failure mode is treated as a correctness problem rather than hidden by a performance average. Error returns carry a concise diagnostic and callers stop the current experiment, preserving the raw evidence needed for debugging.

Verification evidence. The primary check is copy, parse, and record with results. Review should compare configuration, console output, and CSV together, then inspect the git diff against baseline-v3.3.0. This triangulation checks both what the source intends and what the MINIX runtime actually executed.

commands/Makefile

Role and rationale. This unit includes all three experiment command directories. It is kept narrowly scoped so a reviewer can connect the requirement to a small number of functions instead of searching across unrelated MINIX subsystems. Its output or state feeds a documented CSV column, console counter, or downstream correctness check.

Critical review point. The implementation must use native BSD make recursion. The relevant failure mode is treated as a correctness problem rather than hidden by a performance average. Error returns carry a concise diagnostic and callers stop the current experiment, preserving the raw evidence needed for debugging.

Verification evidence. The primary check is top-level command build. Review should compare configuration, console output, and CSV together, then inspect the git diff against baseline-v3.3.0. This triangulation checks both what the source intends and what the MINIX runtime actually executed.

etc/Makefile

Role and rationale. This unit installs all three configuration files. It is kept narrowly scoped so a reviewer can connect the requirement to a small number of functions instead of searching across unrelated MINIX subsystems. Its output or state feeds a documented CSV column, console counter, or downstream correctness check.

Critical review point. The implementation must retain existing file list formatting. The relevant failure mode is treated as a correctness problem rather than hidden by a performance average. Error returns carry a concise diagnostic and callers stop the current experiment, preserving the raw evidence needed for debugging.

Verification evidence. The primary check is make install and /etc inspection. Review should compare configuration, console output, and CSV together, then inspect the git diff against baseline-v3.3.0. This triangulation checks both what the source intends and what the MINIX runtime actually executed.

9. Verification checklist and interpretation guide

Check 1: Confirm the repository HEAD and the commit recorded with the results are identical. Passing this item closes a specific path from implementation error to misleading conclusion. Record the command or artifact used as evidence, the observed result, and the person or timestamp responsible. If the item fails, do not merely tick it after an undocumented change; fix the source or procedure, rerun the affected downstream steps, and preserve the corrected output. This audit trail is especially important when several team members move files between Windows, a shared folder, and the MINIX VM on submission day.

Check 2: Confirm all three binaries report usage errors for unknown command-line options. Passing this item closes a specific path from implementation error to misleading conclusion. Record the command or artifact used as evidence, the observed result, and the person or timestamp responsible. If the item fails, do not merely tick it after an undocumented change; fix the source or procedure, rerun the affected downstream steps, and preserve the corrected output. This audit trail is especially important when several team members move files between Windows, a shared folder, and the MINIX VM on submission day.

Check 3: Confirm configuration parsing rejects zero quantum, zero frames, and inconsistent hierarchy bits. Passing this item closes a specific path from implementation error to misleading conclusion. Record the command or artifact used as evidence, the observed result, and the person or timestamp responsible. If the item fails, do not merely tick it after an undocumented change; fix the source or procedure, rerun the affected downstream steps, and preserve the corrected output. This audit trail is

especially important when several team members move files between Windows, a shared folder, and the MINIX VM on submission day.

Check 4: Confirm every scheduler child exits normally and no worker remains after the command finishes. Passing this item closes a specific path from implementation error to misleading conclusion. Record the command or artifact used as evidence, the observed result, and the person or timestamp responsible. If the item fails, do not merely tick it after an undocumented change; fix the source or procedure, rerun the affected downstream steps, and preserve the corrected output. This audit trail is especially important when several team members move files between Windows, a shared folder, and the MINIX VM on submission day.

Check 5: Confirm the scheduling CSV has one row per algorithm and process combination. Passing this item closes a specific path from implementation error to misleading conclusion. Record the command or artifact used as evidence, the observed result, and the person or timestamp responsible. If the item fails, do not merely tick it after an undocumented change; fix the source or procedure, rerun the affected downstream steps, and preserve the corrected output. This audit trail is especially important when several team members move files between Windows, a shared folder, and the MINIX VM on submission day.

Check 6: Recalculate one turnaround, waiting, and response row manually from arrival, start, and completion. Passing this item closes a specific path from implementation error to misleading conclusion. Record the command or artifact used as evidence, the observed result, and the person or timestamp responsible. If the item fails, do not merely tick it after an undocumented change; fix the source or procedure, rerun the affected downstream steps, and preserve the corrected output. This audit trail is especially important when several team members move files between Windows, a shared folder, and the MINIX VM on submission day.

Check 7: Confirm SJF and priority select only processes whose logical arrival is not in the future. Passing this item closes a specific path from implementation error to misleading conclusion. Record the command or artifact used as evidence, the observed result, and the person or timestamp responsible. If the item fails, do not merely tick it after an undocumented change; fix the source or procedure, rerun the affected downstream steps, and preserve the corrected output. This audit trail is especially important when several team members move files between Windows, a shared folder, and the MINIX VM on submission day.

Check 8: Confirm MLFQ demotes incomplete work and does not index beyond the third queue. Passing this item closes a specific path from implementation error to misleading conclusion. Record the command or artifact used as evidence, the observed result, and the person or timestamp responsible. If the item fails, do not merely tick it after an undocumented change; fix the source or procedure, rerun

the affected downstream steps, and preserve the corrected output. This audit trail is especially important when several team members move files between Windows, a shared folder, and the MINIX VM on submission day.

Check 9: Confirm FIFO load stamps do not change on hits while LRU use stamps do. Passing this item closes a specific path from implementation error to misleading conclusion. Record the command or artifact used as evidence, the observed result, and the person or timestamp responsible. If the item fails, do not merely tick it after an undocumented change; fix the source or procedure, rerun the affected downstream steps, and preserve the corrected output. This audit trail is especially important when several team members move files between Windows, a shared folder, and the MINIX VM on submission day.

Check 10: Confirm first page fills count as faults but not replacements. Passing this item closes a specific path from implementation error to misleading conclusion. Record the command or artifact used as evidence, the observed result, and the person or timestamp responsible. If the item fails, do not merely tick it after an undocumented change; fix the source or procedure, rerun the affected downstream steps, and preserve the corrected output. This audit trail is especially important when several team members move files between Windows, a shared folder, and the MINIX VM on submission day.

Check 11: Confirm empty frames end at $\max(\text{frames minus distinct resident pages}, \text{zero})$. Passing this item closes a specific path from implementation error to misleading conclusion. Record the command or artifact used as evidence, the observed result, and the person or timestamp responsible. If the item fails, do not merely tick it after an undocumented change; fix the source or procedure, rerun the affected downstream steps, and preserve the corrected output. This audit trail is especially important when several team members move files between Windows, a shared folder, and the MINIX VM on submission day.

Check 12: Confirm the same generated byte trace is used when comparing FIFO and LRU. Passing this item closes a specific path from implementation error to misleading conclusion. Record the command or artifact used as evidence, the observed result, and the person or timestamp responsible. If the item fails, do not merely tick it after an undocumented change; fix the source or procedure, rerun the affected downstream steps, and preserve the corrected output. This audit trail is especially important when several team members move files between Windows, a shared folder, and the MINIX VM on submission day.

Check 13: Confirm page-size cases represent the same byte working set and reference count. Passing this item closes a specific path from implementation error to misleading conclusion. Record the command or artifact used as evidence, the observed result, and the person or timestamp responsible. If the item fails, do not merely tick it after an undocumented change; fix the source or procedure, rerun

the affected downstream steps, and preserve the corrected output. This audit trail is especially important when several team members move files between Windows, a shared folder, and the MINIX VM on submission day.

Check 14: Confirm hierarchy index bits plus offset bits equal address width for every case. Passing this item closes a specific path from implementation error to misleading conclusion. Record the command or artifact used as evidence, the observed result, and the person or timestamp responsible. If the item fails, do not merely tick it after an undocumented change; fix the source or procedure, rerun the affected downstream steps, and preserve the corrected output. This audit trail is especially important when several team members move files between Windows, a shared folder, and the MINIX VM on submission day.

Check 15: Confirm real MINIX page-size fields are labelled separately from simulated page size. Passing this item closes a specific path from implementation error to misleading conclusion. Record the command or artifact used as evidence, the observed result, and the person or timestamp responsible. If the item fails, do not merely tick it after an undocumented change; fix the source or procedure, rerun the affected downstream steps, and preserve the corrected output. This audit trail is especially important when several team members move files between Windows, a shared folder, and the MINIX VM on submission day.

Check 16: Confirm the extent device is a scratch device and is not the mounted root or /usr device. Passing this item closes a specific path from implementation error to misleading conclusion. Record the command or artifact used as evidence, the observed result, and the person or timestamp responsible. If the item fails, do not merely tick it after an undocumented change; fix the source or procedure, rerun the affected downstream steps, and preserve the corrected output. This audit trail is especially important when several team members move files between Windows, a shared folder, and the MINIX VM on submission day.

Check 17: Confirm the installed MFS binary was rebuilt from the submitted source before mounting. Passing this item closes a specific path from implementation error to misleading conclusion. Record the command or artifact used as evidence, the observed result, and the person or timestamp responsible. If the item fails, do not merely tick it after an undocumented change; fix the source or procedure, rerun the affected downstream steps, and preserve the corrected output. This audit trail is especially important when several team members move files between Windows, a shared folder, and the MINIX VM on submission day.

Check 18: Confirm `mfs_extent_size` in the mount command equals `extent_blocks` for each controlled case. Passing this item closes a specific path from implementation error to misleading conclusion. Record the command or artifact used as evidence, the observed result, and the person or timestamp

responsible. If the item fails, do not merely tick it after an undocumented change; fix the source or procedure, rerun the affected downstream steps, and preserve the corrected output. This audit trail is especially important when several team members move files between Windows, a shared folder, and the MINIX VM on submission day.

Check 19: Confirm the benchmark directory resolves underneath the scratch mount point. Passing this item closes a specific path from implementation error to misleading conclusion. Record the command or artifact used as evidence, the observed result, and the person or timestamp responsible. If the item fails, do not merely tick it after an undocumented change; fix the source or procedure, rerun the affected downstream steps, and preserve the corrected output. This audit trail is especially important when several team members move files between Windows, a shared folder, and the MINIX VM on submission day.

Check 20: Confirm the MFS startup log prints the requested allocation preference. Passing this item closes a specific path from implementation error to misleading conclusion. Record the command or artifact used as evidence, the observed result, and the person or timestamp responsible. If the item fails, do not merely tick it after an undocumented change; fix the source or procedure, rerun the affected downstream steps, and preserve the corrected output. This audit trail is especially important when several team members move files between Windows, a shared folder, and the MINIX VM on submission day.

Check 21: Confirm unmount prints extent searches, hits, and fallbacks for the completed run. Passing this item closes a specific path from implementation error to misleading conclusion. Record the command or artifact used as evidence, the observed result, and the person or timestamp responsible. If the item fails, do not merely tick it after an undocumented change; fix the source or procedure, rerun the affected downstream steps, and preserve the corrected output. This audit trail is especially important when several team members move files between Windows, a shared folder, and the MINIX VM on submission day.

Check 22: Confirm `verify_errors` is zero in every accepted extent row. Passing this item closes a specific path from implementation error to misleading conclusion. Record the command or artifact used as evidence, the observed result, and the person or timestamp responsible. If the item fails, do not merely tick it after an undocumented change; fix the source or procedure, rerun the affected downstream steps, and preserve the corrected output. This audit trail is especially important when several team members move files between Windows, a shared folder, and the MINIX VM on submission day.

Check 23: Confirm repeated extent trials retain the same total byte count and block size. Passing this item closes a specific path from implementation error to misleading conclusion. Record the command or artifact used as evidence, the observed result, and the person or timestamp responsible. If the item fails,

do not merely tick it after an undocumented change; fix the source or procedure, rerun the affected downstream steps, and preserve the corrected output. This audit trail is especially important when several team members move files between Windows, a shared folder, and the MINIX VM on submission day.

Check 24: Confirm raw CSV files are preserved before graphs or summary calculations are created. Passing this item closes a specific path from implementation error to misleading conclusion. Record the command or artifact used as evidence, the observed result, and the person or timestamp responsible. If the item fails, do not merely tick it after an undocumented change; fix the source or procedure, rerun the affected downstream steps, and preserve the corrected output. This audit trail is especially important when several team members move files between Windows, a shared folder, and the MINIX VM on submission day.

Check 25: Confirm expected/oracle tables are not relabelled as measured MINIX results. Passing this item closes a specific path from implementation error to misleading conclusion. Record the command or artifact used as evidence, the observed result, and the person or timestamp responsible. If the item fails, do not merely tick it after an undocumented change; fix the source or procedure, rerun the affected downstream steps, and preserve the corrected output. This audit trail is especially important when several team members move files between Windows, a shared folder, and the MINIX VM on submission day.

Check 26: Confirm graphs include units, policy names, controlled settings, and an explanatory caption. Passing this item closes a specific path from implementation error to misleading conclusion. Record the command or artifact used as evidence, the observed result, and the person or timestamp responsible. If the item fails, do not merely tick it after an undocumented change; fix the source or procedure, rerun the affected downstream steps, and preserve the corrected output. This audit trail is especially important when several team members move files between Windows, a shared folder, and the MINIX VM on submission day.

Check 27: Confirm failures and outliers remain visible or are excluded only with a recorded reason. Passing this item closes a specific path from implementation error to misleading conclusion. Record the command or artifact used as evidence, the observed result, and the person or timestamp responsible. If the item fails, do not merely tick it after an undocumented change; fix the source or procedure, rerun the affected downstream steps, and preserve the corrected output. This audit trail is especially important when several team members move files between Windows, a shared folder, and the MINIX VM on submission day.

Check 28: Confirm the final archive opens and contains README, source, configs, tests, and scripts. Passing this item closes a specific path from implementation error to misleading conclusion. Record the

command or artifact used as evidence, the observed result, and the person or timestamp responsible. If the item fails, do not merely tick it after an undocumented change; fix the source or procedure, rerun the affected downstream steps, and preserve the corrected output. This audit trail is especially important when several team members move files between Windows, a shared folder, and the MINIX VM on submission day.

Check 29: Confirm the GitHub repository is private until the instructor's publication policy is known. Passing this item closes a specific path from implementation error to misleading conclusion. Record the command or artifact used as evidence, the observed result, and the person or timestamp responsible. If the item fails, do not merely tick it after an undocumented change; fix the source or procedure, rerun the affected downstream steps, and preserve the corrected output. This audit trail is especially important when several team members move files between Windows, a shared folder, and the MINIX VM on submission day.

Check 30: Confirm the submitted report, deck, source archive, and repository commit all agree. Passing this item closes a specific path from implementation error to misleading conclusion. Record the command or artifact used as evidence, the observed result, and the person or timestamp responsible. If the item fails, do not merely tick it after an undocumented change; fix the source or procedure, rerun the affected downstream steps, and preserve the corrected output. This audit trail is especially important when several team members move files between Windows, a shared folder, and the MINIX VM on submission day.

10. Conclusions

The project delivers one coherent experimental framework across three operating-system topics. The scheduling command demonstrates how service order changes individual and average latency even when total CPU demand is fixed. The paging command demonstrates how locality, page geometry, frame capacity, hierarchy overhead, and replacement policy interact. The MFS modification demonstrates how a small change at the bitmap allocation boundary can influence physical placement while preserving file-system format and correctness.

The strongest engineering choice is explicit separation of model, mechanism, and evidence. Real children execute scheduler work, but logical policy time remains reproducible. Configurable page tables are real software data structures, but the report does not misidentify them as x86 hardware mappings. Extent selection changes actual MFS placement, but it does not pretend to introduce a persistent extent tree. These boundaries make the work explainable, testable, and safer to demonstrate.

Final native acceptance is complete. MINIX 3.3.0 built all three commands and the modified MFS server, all smoke tests passed, the repository contains all three raw matrices, every extent verification count is zero, and the captured MFS counters show preferred-run hits without fallbacks for sizes 2 through 32. These artifacts convert the source-complete project into a measured, reproducible submission.

References

1. A. S. Tanenbaum and H. Bos, Modern Operating Systems, 4th ed. Pearson, 2015.
2. A. S. Tanenbaum and A. S. Woodhull, Operating Systems: Design and Implementation, 3rd ed. Pearson, 2006.
3. L. A. Belady, "A Study of Replacement Algorithms for a Virtual-Storage Computer," IBM Systems Journal, vol. 5, no. 2, 1966.
4. P. J. Denning, "The Working Set Model for Program Behavior," Communications of the ACM, vol. 11, no. 5, 1968.
5. F. J. Corbató, M. M. Daggett, and R. C. Daley, "An Experimental Time-Sharing System," AFIPS Spring Joint Computer Conference, 1962.
6. The MINIX 3 Project, "MINIX 3," official project website, <https://www.minix3.org/> (accessed August 18, 2026).
7. MINIX 3.3.0 source tree, local baseline tag baseline-v3.3.0, especially minix/fs/mfs, minix/servers/vm, minix/servers/sched, and system manual pages.
8. CSE335 Project Summer 2026 specification supplied by the course staff.

Appendix A — Result capture sheets

Scheduling evidence record

Record the following fields with the raw artifact: commit; VM version; compiler result; config hash; policy; quantum; average turnaround; average waiting; average response; dispatches; wall time. Store the unedited CSV and console log before calculating any average. State whether the row is measured in MINIX, generated by an independent oracle, or expected from hand calculation. Include enough configuration detail that a reviewer can reproduce the row without guessing a default.

Paging evidence record

Record the following fields with the raw artifact: commit; config hash; trace seed; byte workload; page size; levels and bits; frames; policy; faults; replacements; empty frames; hierarchy bytes; real page size. Store the unedited CSV and console log before calculating any average. State whether the row is

measured in MINIX, generated by an independent oracle, or expected from hand calculation. Include enough configuration detail that a reviewer can reproduce the row without guessing a default.

MFS extents evidence record

Record the following fields with the raw artifact: commit; scratch device; file-system reset method; mount option; benchmark config; repetition; hit/fallback counters; create/write/read/remove time; throughput; verification errors. Store the unedited CSV and console log before calculating any average. State whether the row is measured in MINIX, generated by an independent oracle, or expected from hand calculation. Include enough configuration detail that a reviewer can reproduce the row without guessing a default.